Results of Methods – DigitalNumbers

Jerónimo Mesa Alzate - Juan Guillermo Isaza - Felipe Florez - Luciana Pineda

April 20th 2626

Chapter 1

Single Variable Methods

1.1 Incremental Search

1.1.1 Method Overview

The incremental search method finds an interval $[a, b]$ where $f(x)$ changes sign, guaranteeing (by the Intermediate Value Theorem) that a root exists inside it. Starting from x_0, the function is evaluated at successive points $x_0, x_0+h, x_0+2h, \ldots$ until a sign change is detected.

1.1.2 Test Case 1 — $f(x) = \cos(x) - x$

$$f(x) = \cos(x) - x, \qquad x_0 = 0.0, \quad h = 0.5, \quad N = 100$$

Table 1.1: Incremental Search — $f(x) = \cos(x) - x$

Iteration	x_{prev}	$f(x_{\text{prev}})$	x_{curr}	$f(x_{\text{curr}})$
1	0.0	1.000000	0.5	0.377583
2	0.5	0.377583	1.0	-0.459698

Result: Sign change found in interval [0.5, 1.0] after 2 iterations.

1.1.3 Test Case 2 — $f(x) = x^3 - x - 2$

$$f(x) = x^3 - x - 2, \qquad x_0 = 0.0, \quad h = 0.5, \quad N = 100$$

Table 1.2: Incremental Search — $f(x) = x^3 - x - 2$

Iteration	x_{prev}	$f(x_{\text{prev}})$	x_{curr}	$f(x_{\text{curr}})$
1	0.0	-2.000	0.5	-2.375
2	0.5	-2.375	1.0	-2.000
3	1.0	-2.000	1.5	-0.125
4	1.5	-0.125	2.0	4.000

Result: Sign change found in interval [1.5, 2.0] after 4 iterations.

1.2 Bisection

1.2.1 Method Overview

Given a bracket $[a, b]$ with $f(a) \cdot f(b) < 0$, the bisection method repeatedly halves the interval and retains the half where the sign change persists, until the interval width falls below the tolerance ε. The midpoint $p = (a + b)/2$ is the root approximation at each step.

1.2.2 Test Case 1 — $f(x) = \cos(x) - x$

$$f(x) = \cos(x) - x, \qquad [a, b] = [0,\ 1], \quad \varepsilon = 10^{-7}, \quad N = 100$$

Table 1.3: Bisection — $f(x) = \cos(x) - x$

Iter.	a	b	midpoint	f(mid)	error
1	0.000000	1.000000	0.500000	3.776e-01	1.000e+03
2	0.500000	1.000000	0.750000	-1.831e-02	2.500e-01
3	0.500000	0.750000	0.625000	1.860e-01	1.250e-01
4	0.625000	0.750000	0.687500	8.533e-02	6.250e-02
5	0.687500	0.750000	0.718750	3.388e-02	3.125e-02
6	0.718750	0.750000	0.734375	7.875e-03	1.563e-02
7	0.734375	0.750000	0.742188	-5.196e-03	7.813e-03
8	0.734375	0.742188	0.738281	1.345e-03	3.906e-03
9	0.738281	0.742188	0.740234	-1.924e-03	1.953e-03
10	0.738281	0.740234	0.739258	-2.890e-04	9.766e-04
11	0.738281	0.739258	0.738770	5.282e-04	4.883e-04
12	0.738770	0.739258	0.739014	1.196e-04	2.441e-04
13	0.739014	0.739258	0.739136	-8.470e-05	1.221e-04
14	0.739014	0.739136	0.739075	1.745e-05	6.104e-05
15	0.739075	0.739136	0.739105	-3.363e-05	3.052e-05
16	0.739075	0.739105	0.739090	-8.088e-06	1.526e-05
17	0.739075	0.739090	0.739082	4.681e-06	7.629e-06
18	0.739082	0.739090	0.739086	-1.704e-06	3.815e-06
19	0.739082	0.739086	0.739084	1.489e-06	1.907e-06
20	0.739084	0.739086	0.739085	-1.075e-07	9.537e-07
21	0.739084	0.739085	0.739085	6.905e-07	4.768e-07
22	0.739085	0.739085	0.739085	2.915e-07	2.384e-07
23	0.739085	0.739085	0.739085	9.200e-08	1.192e-07
24	0.739085	0.739085	0.739085	-7.700e-09	5.960e-08

Result: Root ≈ 0.7390851378, Error $= 5.96 \times 10^{-8}$, Iterations: 24. *Converged*

1.2.3 Test Case 2 — $f(x) = x^3 - x - 2$

$$f(x) = x^3 - x - 2, \qquad [a, b] = [1,\ 2], \quad \varepsilon = 10^{-7}, \quad N = 100$$

Table 1.4: Bisection — $f(x) = x^3 - x - 2$

Iter.	a	b	midpoint	f(mid)	error
1	1.000000	2.000000	1.500000	-1.250e-01	1.000e+03
2	1.500000	2.000000	1.750000	1.609e+00	2.500e-01
3	1.500000	1.750000	1.625000	6.660e-01	1.250e-01
4	1.500000	1.625000	1.562500	2.522e-01	6.250e-02
5	1.500000	1.562500	1.531250	5.911e-02	3.125e-02
6	1.500000	1.531250	1.515625	-3.405e-02	1.563e-02
7	1.515625	1.531250	1.523438	1.225e-02	7.813e-03
8	1.515625	1.523438	1.519531	-1.097e-02	3.906e-03
9	1.519531	1.523438	1.521484	6.222e-04	1.953e-03
10	1.519531	1.521484	1.520508	-5.179e-03	9.766e-04
11	1.520508	1.521484	1.520996	-2.279e-03	4.883e-04
12	1.520996	1.521484	1.521240	-8.289e-04	2.441e-04
13	1.521240	1.521484	1.521362	-1.034e-04	1.221e-04
14	1.521362	1.521484	1.521423	2.594e-04	6.104e-05
15	1.521362	1.521423	1.521393	7.796e-05	3.052e-05
16	1.521362	1.521393	1.521378	-1.274e-05	1.526e-05
17	1.521378	1.521393	1.521385	3.261e-05	7.629e-06
18	1.521378	1.521385	1.521381	9.934e-06	3.815e-06
19	1.521378	1.521381	1.521379	-1.403e-06	1.907e-06
20	1.521379	1.521381	1.521380	4.266e-06	9.537e-07
21	1.521379	1.521380	1.521380	1.432e-06	4.768e-07
22	1.521379	1.521380	1.521380	1.450e-08	2.384e-07
23	1.521379	1.521380	1.521380	-6.941e-07	1.192e-07
24	1.521380	1.521380	1.521380	-3.398e-07	5.960e-08

Result: Root ≈ 1.5213796496, Error $= 5.96 \times 10^{-8}$, Iterations: 24. *Converged*

1.3 False Position (Regula Falsi)

1.3.1 Method Overview

The false position method improves on bisection by replacing the midpoint with a linearly interpolated estimate. Given $[a, b]$ with opposite signs, the new estimate is:

$$p = \frac{f(b) \cdot a - f(a) \cdot b}{f(b) - f(a)}$$

The interval is then updated exactly as in bisection. This generally converges faster when the function is smooth.

1.3.2 Test Case 1 — $f(x) = \cos(x) - x$

$$f(x) = \cos(x) - x, \qquad [a, b] = [0,\ 1], \quad \varepsilon = 10^{-7}, \quad N = 100$$

Table 1.5: False Position — $f(x) = \cos(x) - x$

Iter.	a	b	p	$f(p)$	error
1	0.000000	1.0	0.685073	8.930e-02	1.000e+03
2	0.685073	1.0	0.736299	4.660e-03	5.123e-02
3	0.736299	1.0	0.738945	2.339e-04	2.646e-03
4	0.738945	1.0	0.739078	1.172e-05	1.328e-04
5	0.739078	1.0	0.739085	5.870e-07	6.652e-06
6	0.739085	1.0	0.739085	2.940e-08	3.332e-07
7	0.739085	1.0	0.739085	1.500e-09	1.670e-08

Result: Root ≈ 0.7390851323, Error $= 1.67 \times 10^{-8}$, Iterations: 7. *Converged*

Note the significant improvement over bisection: 7 iterations vs. 24 for the same tolerance, due to the linear interpolation better approximating the root when f is nearly linear near the solution.

1.3.3 Test Case 2 — $f(x) = x^3 - x - 2$

$$f(x) = x^3 - x - 2, \qquad [a, b] = [1,\ 2], \quad \varepsilon = 10^{-7}, \quad N = 100$$

Table 1.6: False Position — $f(x) = x^3 - x - 2$

Iter.	a	b	p	$f(p)$	error
1	1.000000	2.0	1.333333	-9.630e-01	1.000e+03
2	1.333333	2.0	1.462687	-3.333e-01	1.294e-01
3	1.462687	2.0	1.504019	-1.018e-01	4.133e-02
4	1.504019	2.0	1.516331	-2.989e-02	1.231e-02
5	1.516331	2.0	1.519919	-8.675e-03	3.588e-03
6	1.519919	2.0	1.520957	-2.509e-03	1.039e-03
7	1.520957	2.0	1.521258	-7.248e-04	3.003e-04
8	1.521258	2.0	1.521344	-2.093e-04	8.674e-05
9	1.521344	2.0	1.521370	-6.046e-05	2.505e-05
10	1.521370	2.0	1.521377	-1.746e-05	7.235e-06
11	1.521377	2.0	1.521379	-5.043e-06	2.089e-06
12	1.521379	2.0	1.521379	-1.456e-06	6.034e-07
13	1.521379	2.0	1.521380	-4.206e-07	1.743e-07
14	1.521380	2.0	1.521380	-1.215e-07	5.030e-08

Result: Root ≈ 1.5213796864, Error $= 5.03 \times 10^{-8}$, Iterations: 14. *Converged*

For this function, false position required 14 iterations vs. 24 for bisection — again faster, though the asymmetry of f on $[1, 2]$ slows convergence compared to the first test case.

1.4 Newton-Raphson

1.4.1 Method Overview

The Newton-Raphson method is an open root-finding method that uses the tangent line at the current approximation to estimate the next point.

Given an initial guess x_0, the iteration formula is:

$$x_{n+1} = x_n - \frac{f(x_n)}{f'(x_n)}$$

This method typically converges very fast when the initial guess is close to the root and the function is well-behaved.

1.4.2 Test Case — $f(x) = \ln(\sin^2(x) + 1) - 0.5$

$$f(x) = \ln(\sin^2(x) + 1) - 0.5, \qquad x_0 = 0.5, \quad \varepsilon = 10^{-7}, \quad N = 100$$

Table 1.7: Newton-Raphson — $f(x) = \ln(\sin^2(x) + 1) - 0.5$

Iter.	x_i	$f(x_i)$	Error
0	0.5000000000	-2.9e-01	
1	0.9283919899	-4.7e-03	4.3e-01
2	0.9363667413	-2.2e-05	8.0e-03
3	0.9364045800	-5.0e-10	3.8e-05
4	0.9364045809	-1.1e-16	8.6e-10

Result: Root ≈ 0.9364045809, Error $= 8.6 \times 10^{-10}$, Iterations: 4. *Converged*

1.5 Fixed-Point Iteration

1.5.1 Method Overview

The fixed-point method transforms the equation $f(x) = 0$ into the form $x = g(x)$ and iteratively computes:

$$x_{n+1} = g(x_n)$$

The method converges when $|g'(x)| < 1$ near the root. Convergence speed depends strongly on the choice of the function $g(x)$.

1.5.2 Test Case — Reformulated function

$$f(x) = x - \big(\ln(\sin^2(x) + 1) - 0.5\big)$$

$$g(x) = \ln(\sin^2(x) + 1) - 0.5, \qquad x_0 = -0.5, \quad \varepsilon = 10^{-7}, \quad N = 100$$

Table 1.8: Fixed-Point Iteration

Iter.	x_i	$g(x_i)$	$f(x_i)$	Error
0	-0.5000000000	-0.2931087267	-2.1e-01	
1	-0.2931087267	-0.4198215436	1.3e-01	2.1e-01
2	-0.4198215436	-0.3463045192	-7.4e-02	1.3e-01
3	-0.3463045192	-0.3909584565	4.5e-02	7.4e-02
4	-0.3909584565	-0.3644050349	-2.7e-02	4.5e-02
5	-0.3644050349	-0.3804263032	1.6e-02	2.7e-02
6	-0.3804263032	-0.3708367953	-9.6e-03	1.6e-02
7	-0.3708367953	-0.3766056454	5.8e-03	9.6e-03
8	-0.3766056454	-0.3731454176	-3.5e-03	5.8e-03
9	-0.3731454176	-0.3752246412	2.1e-03	3.5e-03
10	-0.3752246412	-0.3739765860	-1.2e-03	2.1e-03
11	-0.3739765860	-0.3747262157	7.5e-04	1.2e-03
12	-0.3747262157	-0.3742761333	-4.5e-04	7.5e-04
13	-0.3742761333	-0.3745464285	2.7e-04	4.5e-04
14	-0.3745464285	-0.3743841264	-1.6e-04	2.7e-04
15	-0.3743841264	-0.3744815908	9.7e-05	1.6e-04
16	-0.3744815908	-0.3744230652	-5.9e-05	9.7e-05
17	-0.3744230652	-0.3744582099	3.5e-05	5.9e-05
18	-0.3744582099	-0.3744371058	-2.1e-05	3.5e-05
19	-0.3744371058	-0.3744497787	1.3e-05	2.1e-05
20	-0.3744497787	-0.3744421688	-7.6e-06	1.3e-05
21	-0.3744421688	-0.3744467385	4.6e-06	7.6e-06
22	-0.3744467385	-0.3744439944	-2.7e-06	4.6e-06
23	-0.3744439944	-0.3744456422	1.6e-06	2.7e-06
24	-0.3744456422	-0.3744446527	-9.9e-07	1.6e-06
25	-0.3744446527	-0.3744452469	5.9e-07	9.9e-07
26	-0.3744452469	-0.3744448901	-3.6e-07	5.9e-07
27	-0.3744448901	-0.3744451044	2.1e-07	3.6e-07
28	-0.3744451044	-0.3744449757	-1.3e-07	2.1e-07
29	-0.3744449757	-0.3744450530	7.7e-08	1.3e-07
30	-0.3744450530	-0.3744450066	-4.6e-08	7.7e-08

Result: Root ≈ -0.37444505296, Error $= 7.7 \times 10^{-8}$, Iterations: 30. *Converged*

1.6 Secant Method

1.6.1 Method Overview

The secant method approximates a root of a nonlinear function using two initial guesses and successive linear approximations without requiring derivatives. At each iteration, a new estimate is computed as the intersection of the secant line with the x-axis.

1.6.2 Test Case — $f(x) = \ln(\sin^2(x) + 1) - \frac{1}{2}$

$$f(x) = \ln(\sin^2(x) + 1) - \frac{1}{2}, \qquad x_0 = 0.5,\ x_1 = 1, \quad \varepsilon = 10^{-7}, \quad N = 100$$

Table 1.9: Secant Method — $f(x) = \ln(\sin^2(x) + 1) - \frac{1}{2}$

Iter.	x_i	$f(x_i)$	error
1	0.946166	5.619393e-03	5.383378e-02
2	0.935997	-2.363222e-04	1.016964e-02
3	0.936407	1.402236e-06	4.104216e-04
4	0.936405	3.437165e-10	2.420904e-06
5	0.936405	-4.996004e-16	5.935581e-10

Result: Root ≈ 0.9364045809, Error $= 5.94 \times 10^{-10}$, Iterations: 5. *Converged*

1.7 Multiple Roots Method

1.7.1 Method Overview

The multiple roots method is a modification of Newton's method designed to improve convergence when the root has multiplicity greater than one. It uses first and second derivatives to adjust the iteration formula and avoid slow convergence.

1.7.2 Test Case — $h(x) = e^x - x - 1$

$$h(x) = e^x - x - 1, \qquad x_0 = 1, \quad \varepsilon = 10^{-7}, \quad N = 100$$

Table 1.10: Multiple Roots Method — $h(x) = e^x - x - 1$

Iter.	x_i	$f(x_i)$	error
1	1.000000e+00	7.182818e-01	1.234211
2	-2.342106e-01	2.540578e-02	0.225752
3	-8.458280e-03	3.567061e-05	0.008446
4	-1.189018e-05	7.068790e-11	0.000012
5	-4.218591e-11	0.000000e+00	0.000000

Result: Root ≈ 0.0000000000, Error $= 0.00 \times 10^{0}$, Iterations: 5. *Converged*

Chapter 2

Linear Systems

2.1 Naive Gaussian Elimination

2.1.1 Test Case: 4×4 System

The following system $A\mathbf{x} = \mathbf{b}$ was used as input:

$$A = \begin{pmatrix} 2 & -1 & 0 & 3 \\ 1 & 0.5 & 3 & 8 \\ 0 & 13 & -2 & 11 \\ 14 & 5 & -2 & 3 \end{pmatrix}, \qquad \mathbf{b} = \begin{pmatrix} 1 \\ 1 \\ 1 \\ 1 \end{pmatrix}$$

2.1.2 Forward Elimination Stages

The augmented matrix $[A \mid \mathbf{b}]$ is shown at each elimination stage below.

Stage 0 — Initial augmented matrix

$$\left(\begin{array}{rrrr|r} 2 & -1 & 0 & 3 & 1 \\ 1 & 0.5 & 3 & 8 & 1 \\ 0 & 13 & -2 & 11 & 1 \\ 14 & 5 & -2 & 3 & 1 \end{array}\right)$$

Stage 1 — After eliminating column 1

$$\left(\begin{array}{rrrr|r} 2 & -1 & 0 & 3 & 1 \\ 0 & 1 & 3 & 6.5 & 0.5 \\ 0 & 13 & -2 & 11 & 1 \\ 0 & 12 & -2 & -18 & -6 \end{array}\right)$$

Stage 2 — After eliminating column 2

$$\left(\begin{array}{rrrr|r} 2 & -1 & 0 & 3 & 1 \\ 0 & 1 & 3 & 6.5 & 0.5 \\ 0 & 0 & -41 & -73.5 & -5.5 \\ 0 & 0 & -38 & -96 & -12 \end{array}\right)$$

Stage 3 — Upper triangular form

$$\left(\begin{array}{rrrr|r} 2 & -1 & 0 & 3 & 1 \\ 0 & 1 & 3 & 6.5 & 0.5 \\ 0 & 0 & -41 & -73.5 & -5.5 \\ 0 & 0 & 0 & -27.878049 & -6.902439 \end{array}\right)$$

2.1.3 Solution via Back Substitution

After applying back substitution to the upper triangular system, the solution vector $\mathbf{x}$ is:

$$\mathbf{x} = \begin{pmatrix} x_1 \\ x_2 \\ x_3 \\ x_4 \end{pmatrix} = \begin{pmatrix} 0.038495 \\ -0.180227 \\ -0.309711 \\ 0.247594 \end{pmatrix}$$

2.1.4 GitHub Actions Verification

The following screenshot confirms that the implementation was successfully executed via a GitHub Actions workflow in the project repository.

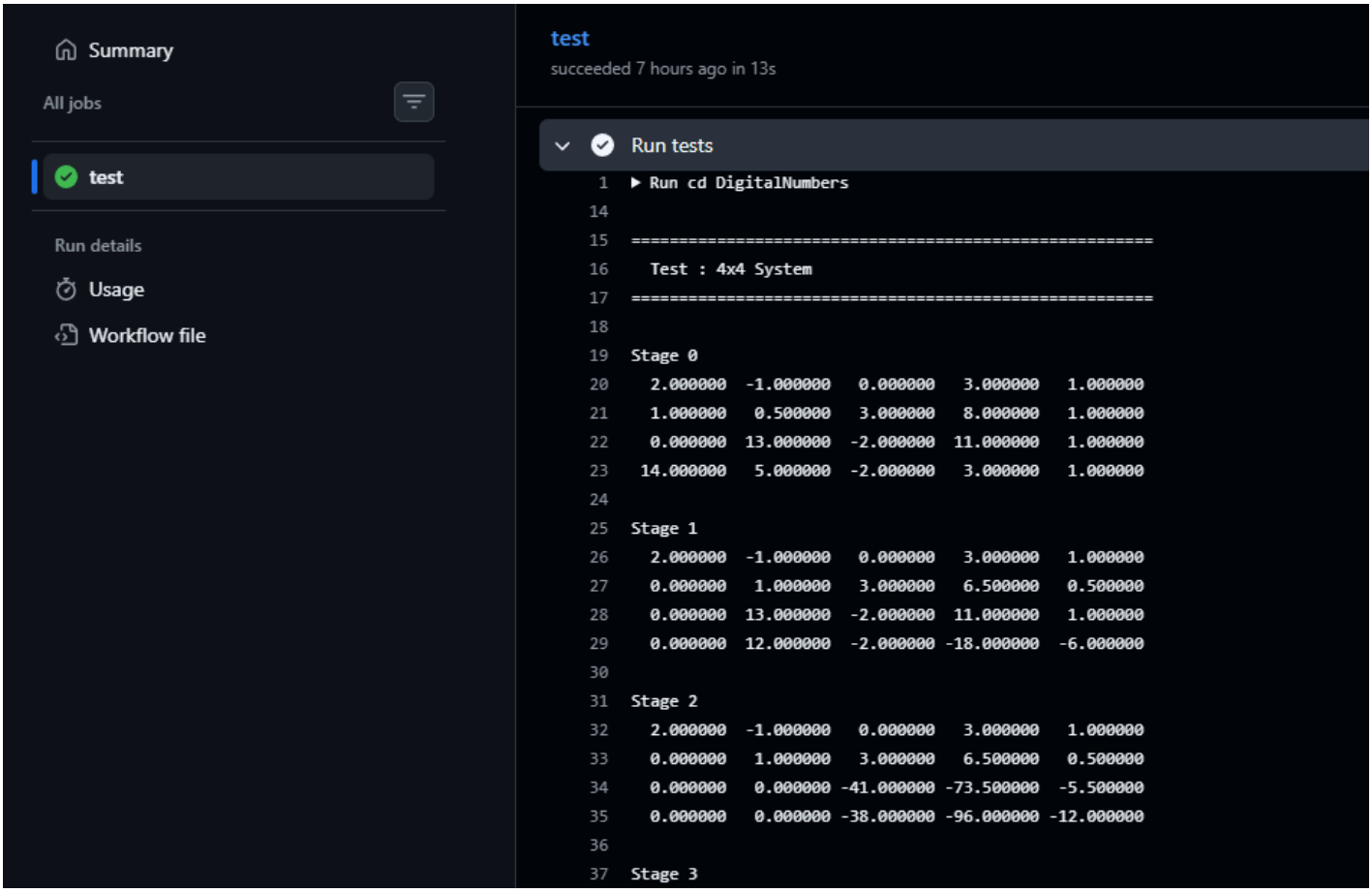

Figure 2.1: GitHub Actions run — `test` job succeeded (4×4 system test passed).

2.2 Gaussian Elimination with Partial Pivoting

2.2.1 Test Case: 4×4 System

The same system $A\mathbf{x} = \mathbf{b}$ from Section 2.1 was used as input.

2.2.2 Forward Elimination Stages

At each stage, the row with the largest absolute value in the pivot column is swapped to the pivot position before elimination.

Stage 0 — Initial augmented matrix

$$\left(\begin{array}{rrrr|r} 2 & -1 & 0 & 3 & 1 \\ 1 & 0.5 & 3 & 8 & 1 \\ 0 & 13 & -2 & 11 & 1 \\ 14 & 5 & -2 & 3 & 1 \end{array}\right)$$

Stage 1 — Pivot: row swap to bring $|14|$ to top

$$\left(\begin{array}{rrrr|r} 14 & 5 & -2 & 3 & 1 \\ 0 & 0.142857 & 3.142857 & 7.785714 & 0.928571 \\ 0 & 13 & -2 & 11 & 1 \\ 0 & -1.714286 & 0.285714 & 2.571429 & 0.857143 \end{array}\right)$$

Stage 2 — Pivot: row swap to bring $|13|$ to position (2,2)

$$\left(\begin{array}{rrrr|r} 14 & 5 & -2 & 3 & 1 \\ 0 & 13 & -2 & 11 & 1 \\ 0 & 0 & 3.164835 & 7.664835 & 0.917582 \\ 0 & 0 & 0.021978 & 4.021978 & 0.989011 \end{array}\right)$$

Stage 3 — Upper triangular form

$$\left(\begin{array}{rrrr|r} 14 & 5 & -2 & 3 & 1 \\ 0 & 13 & -2 & 11 & 1 \\ 0 & 0 & 3.164835 & 7.664835 & 0.917582 \\ 0 & 0 & 0 & 3.968750 & 0.982639 \end{array}\right)$$

2.2.3 Solution via Back Substitution

$$\mathbf{x} = \begin{pmatrix} x_1 \\ x_2 \\ x_3 \\ x_4 \end{pmatrix} = \begin{pmatrix} 0.038495 \\ -0.180227 \\ -0.309711 \\ 0.247594 \end{pmatrix}$$

2.2.4 GitHub Actions Verification

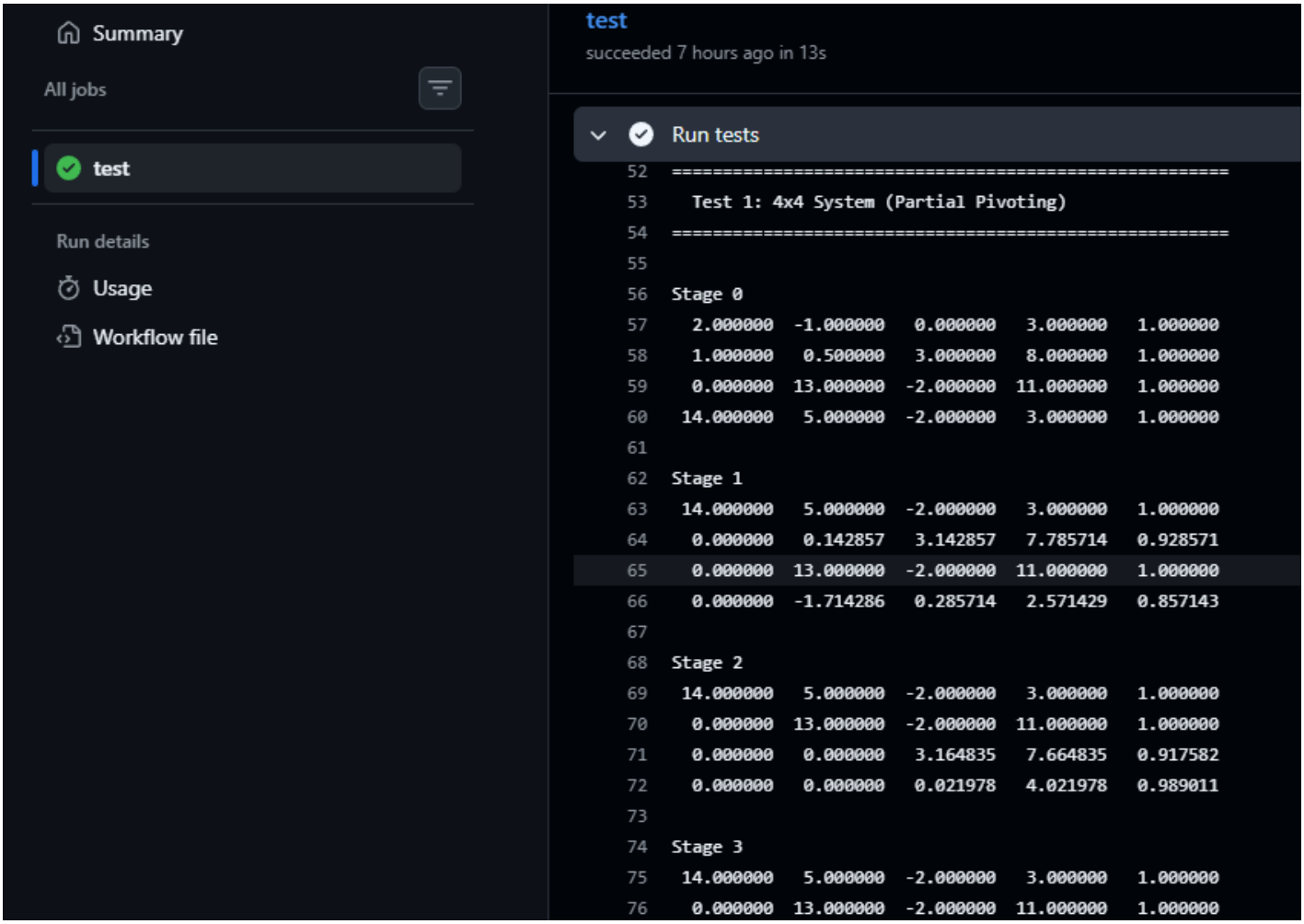

Figure 2.2: GitHub Actions run — Gaussian elimination with partial pivoting test passed.

2.3 Gaussian Elimination with Total Pivoting

2.3.1 Test Case: 4×4 System

The same system $A\mathbf{x} = \mathbf{b}$ from Section 2.1 was used as input.

2.3.2 Forward Elimination Stages

At each stage, the element with the largest absolute value in the entire remaining submatrix is selected as pivot, swapping both rows and columns as needed.

Stage 0 — Initial augmented matrix

$$\left(\begin{array}{rrrr|r} 2 & -1 & 0 & 3 & 1 \\ 1 & 0.5 & 3 & 8 & 1 \\ 0 & 13 & -2 & 11 & 1 \\ 14 & 5 & -2 & 3 & 1 \end{array}\right)$$

Stage 1 — Pivot: row swap to bring $|14|$ to top

$$\left(\begin{array}{rrrr|r} 14 & 5 & -2 & 3 & 1 \\ 0 & 0.142857 & 3.142857 & 7.785714 & 0.928571 \\ 0 & 13 & -2 & 11 & 1 \\ 0 & -1.714286 & 0.285714 & 2.571429 & 0.857143 \end{array}\right)$$

Stage 2 — Pivot: row swap to bring $|13|$ to position (2,2)

$$\left(\begin{array}{rrrr|r} 14 & 5 & -2 & 3 & 1 \\ 0 & 13 & -2 & 11 & 1 \\ 0 & 0 & 3.164835 & 7.664835 & 0.917582 \\ 0 & 0 & 0.021978 & 4.021978 & 0.989011 \end{array}\right)$$

Stage 3 — Column swap to bring $|7.664835|$ to pivot position (3,3)

$$\left(\begin{array}{rrrr|r} 14 & 5 & 3 & -2 & 1 \\ 0 & 13 & 11 & -2 & 1 \\ 0 & 0 & 7.664835 & 3.164835 & 0.917582 \\ 0 & 0 & 0 & -1.638710 & 0.507527 \end{array}\right)$$

2.3.3 Solution via Back Substitution

The column permutation is accounted for during back substitution, yielding the same solution:

$$\mathbf{x} = \begin{pmatrix} x_1 \\ x_2 \\ x_3 \\ x_4 \end{pmatrix} = \begin{pmatrix} 0.038495 \\ -0.180227 \\ -0.309711 \\ 0.247594 \end{pmatrix}$$

2.3.4 GitHub Actions Verification

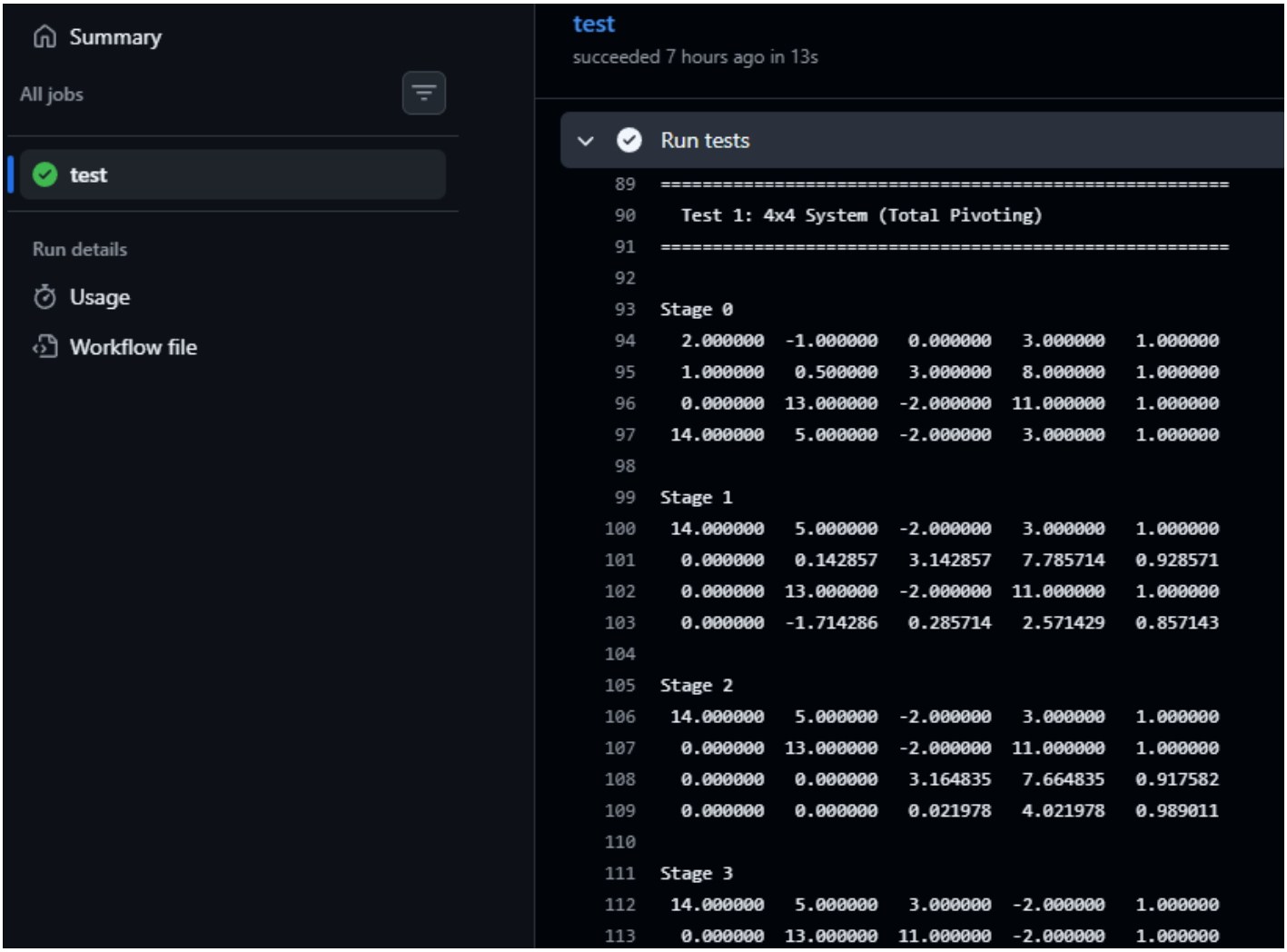

Figure 2.3: GitHub Actions run — Gaussian elimination with total pivoting test passed.